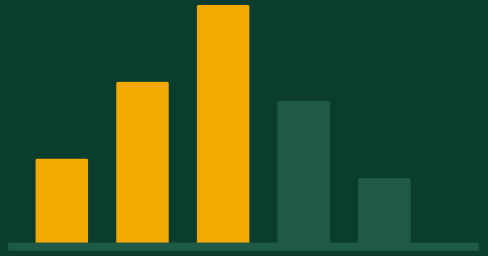
How loud is this room?

Lesson 6 - A meter that remembers



JA05 STEAM and Service Learning - 30 Oct 2026

Who does noise hurt?

Hard of hearing

A hearing aid makes everything louder, the noise included.

Trouble focusing

Some people cannot filter a room out. The noise is all of it.

The teacher

Talking over a loud room all day. Voices give out.

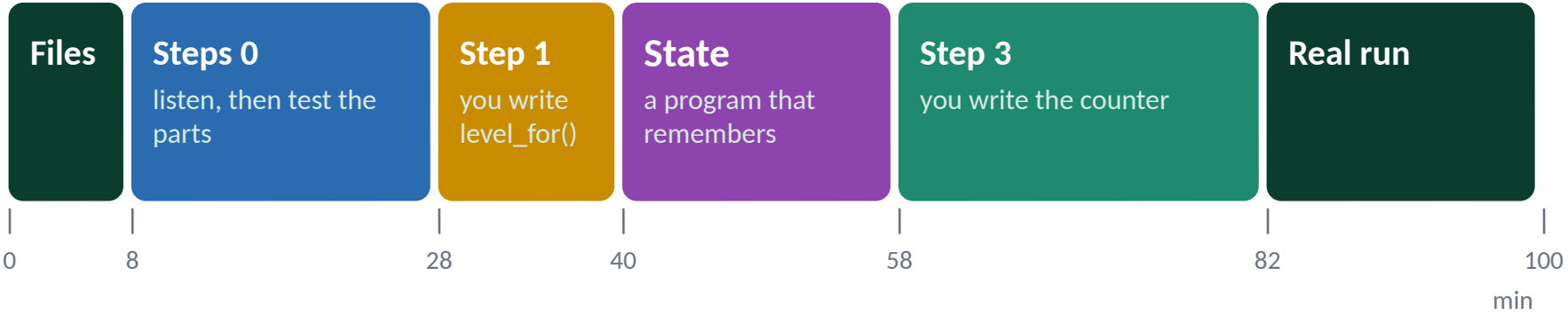
Everybody

You stop hearing it. You do not stop being tired by it.

Nobody wins an argument about noise. Somebody says it is fine, somebody says it is not.

A number ends the argument. Today you build the thing that makes the number.

Today: 100 minutes



Three new ideas today: measuring something that will not hold still, a program that remembers, and how to find a bug on purpose instead of by luck.

CHECKPOINT By minute 40 every group has an LED bar that follows their voice.

Get today's files

```
Terminal
MacBook:~ s20xxxxxx$ cd ~/JA05
MacBook:JA05 s20xxxxxx$ ./get_latest.sh
Checking GitHub for new files...
Done. You have the latest files.

New folder: 20261030_L6
```

1. Saves anything you changed into my_work
2. Gets the newest files from GitHub
3. Never touches your ESP32

CHECKPOINT A folder 20261030_L6 appears in JA05.

STUCK? permission denied? Type: bash get_latest.sh

Step 4 also needs lcd128_32.py and lcd128_32_fonts.py uploaded. They are in the folder.

The seven files, in order

	you do	you should see
`step0_read_sound.py`	RUN	The number jumps when you clap
`step0b_test_parts.py`	RUN	All three tests pass
`step1_peak_level.py`	WRITE 1 line	The LED bar follows your voice
`step2_counter_demo.py`	RUN	One count sticks at 1, the other counts up
`step2b_find_the_bug.py`	MOVE 1 line	It prints 3
`step3_noise_counter.py`	WRITE 2 lines	One clap adds exactly 1
`step4_lcd_report.py`	RUN, if you get there	The screen matches the LEDs

Every file says WHAT YOU DO at the top. RUN means run it unchanged: reading it IS the work.

Every file has the same five parts

1. **SETUP** — the pins and the sensor — *lines 20-30*
2. **NUMBERS YOU CAN CHANGE** — QUIET, LOUD, COOLDOWN — *lines 33-40*
3. **FUNCTIONS** — the thinking: loudness(), level_for() — *lines 43-68*
4. **THE MEMORY** — made once, before the loop — *lines 70-75*
5. **THE LOOP** — again and again, until you stop it — *lines 77-100*

line numbers from step3_noise_counter.py

You only ever touch two parts

A function in part 3, or the loop in part 5. Parts 1 and 4 you read, never edit.

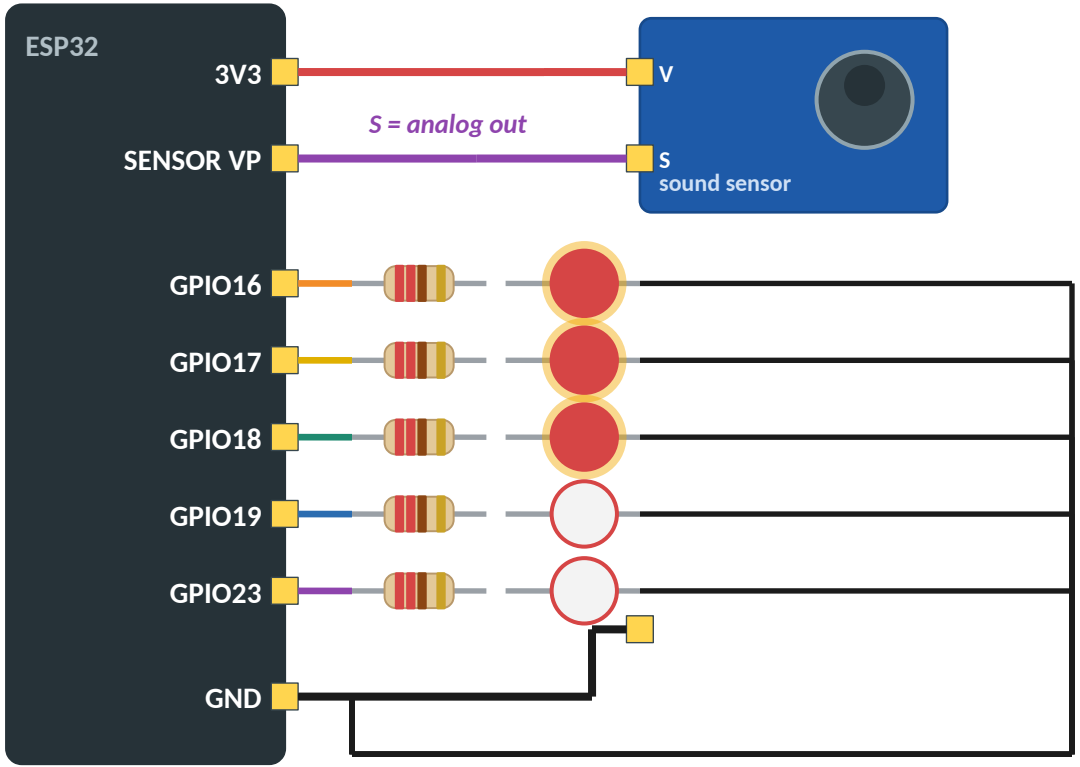
Look for these

```
>>>>> YOUR CODE
      STARTS HERE >>>>>

<<<<< YOUR CODE
      ENDS HERE <<<<<
```

Scroll until you see the arrows. Your work goes between them, and nowhere else.

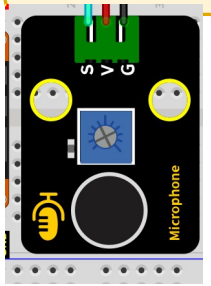
Wire it: microphone + Lesson 3 LEDs



The blue screw

sets how big the signal reaching S is. It does NOT set what counts as loud: your code does that. Set it once, leave it.

Only these pins today.
Leave the ultrasonic sensor, the buzzer and the servo in the box.



`S`, `V`, `G` in that order on the module.
SENSOR VP is the pin under EN.

Step 0: run it and listen

0

step0_read_sound.py · lines 29–37

```
29  mic.width(ADC.WIDTH_12BIT)     # answer as 0 to 4095
30
31  try:
  ⋮
34          print(value)
35          time.sleep(0.2)
36
37  finally:
```

CHECKPOINT The number jumps when you clap.

STUCK? Always 0? S is on SENSOR VP, the pin under EN.

ATTN_11DB means measure the whole 0 to 3.3 V. WIDTH_12BIT means answer as 0 to 4095.

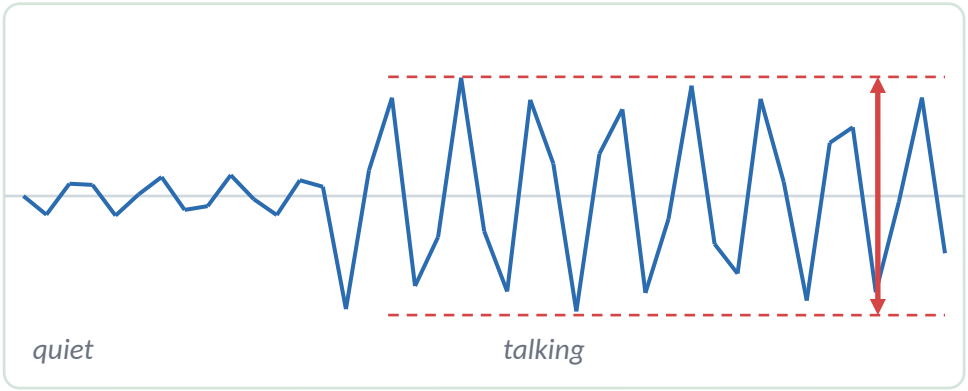
Terminal

```
quiet room
  6   11   4   9   7   5
  8    6   12  5   9   6

one clap
  7   1240   843   96
  310   44    9     7
```

Your numbers will not match these. They depend on the gain screw. That is the point of Step 0b.

One reading lies



A single reading is a photograph of a moving thing. You need the whole swing.

what A microphone reports where its diaphragm is, right now

loud means it swings further, not that the number gets bigger

so take 50 readings fast. Loudest minus quietest is the loudness

Step 0b: run the three part tests

0b

Terminal

```
TEST 1 of 3: the LEDs, one at a time.
  LED on GPIO 16
  LED on GPIO 17
  ...

TEST 2 of 3: the sensor, in silence.
  lowest: 4  highest: 37  gap: 33

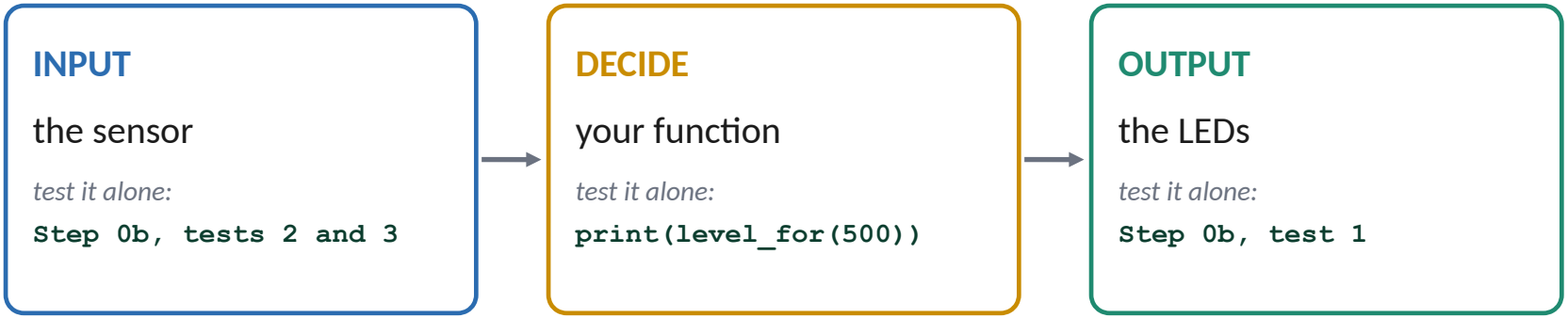
TEST 3 of 3: the sensor, with noise.
  lowest: 5  highest: 1690  gap: 1685
```

1. All 5 LEDs lit? If not, that LED or its resistor
2. Quiet gap under about 60? If not, turn the screw back
3. Clap gap in the hundreds? If not, check S, then the screw

CHECKPOINT All three tests pass.

Pass all three, and any bug after this is in your code, not your wiring.

Test the parts, not the whole



You can test the middle box with no sensor and no noise at all, because it is a function. That is what functions are for.

Three parts, three tests. A broken whole becomes one broken part, and a broken part is a wire you can look at.

Step 1: you write level_for()

1

step1_peak_level.py · lines 53–60

```
53  def level_for(gap):
54      """Turn a loudness gap into how many LEDs: 0 to 5."""
55      # >>>>> YOUR CODE STARTS HERE >>>>>
56      # One line. Under QUIET -> 0 LEDs. Over LOUD -> 5 LEDs.
57      # In between, spread it out. Same shape as leds_for()
58      # in Lesson 3 and angle_for() in Lesson 5.
59      return 0
60      # <<<<< YOUR CODE ENDS HERE <<<<<
```

loudness() is given. It hands level_for() one number.

step1_peak_level.py · lines 40–50

```
40  def loudness():
41      """SAMPLES readings fast. Loudest minus quietest."""
  ⋮
49              highest = value
50      return highest - lowest
```

quiet gap under QUIET: 0 LEDs

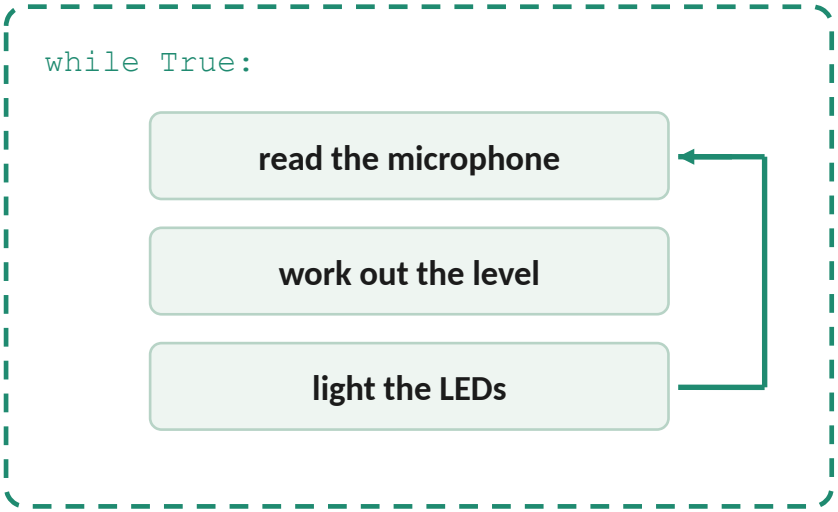
loud gap over LOUD: 5 LEDs

between spread it across 1 to 4

CHECKPOINT The bar follows your voice.

STUCK? All 5 lit in silence? QUIET is too low for this room.

A loop forgets



round and round, ten times a second

Every round starts the same way

The loop does not know this is round 400. It does not know the room was loud a second ago. It knows nothing except what you stored.

State

What the program knows right now, because of what happened before. A loop has no memory of its own. You give it one.

Inside or outside the loop?

WRONG

```
while True:
    count = 0
    count = count + 1
    print(count)
```

Terminal

```
1
1
1
1
1
```

RIGHT

```
count = 0
while True:
    count = count + 1
    print(count)
```

Terminal

```
1
2
3
4
5
```

One line moved. Everything changed.

Step 2: run it and watch

2

step2_counter_demo.py · lines 17–28

```
17  print("WRONG: count is made again every time round the loop")
18  for round_number in range(1, 6):
19      count = 0            # made again, old value lost
20      count = count + 1
21      print("  round", round_number, "count is", count)
    ⋮
24  print("RIGHT: count is made ONCE, before the loop")
25  count = 0                # made once, the loop adds to it
26  for round_number in range(1, 6):
27      count = count + 1
28      print("  round", round_number, "count is", count)
```

CHECKPOINT The first count never passes 1. The second counts to 5.

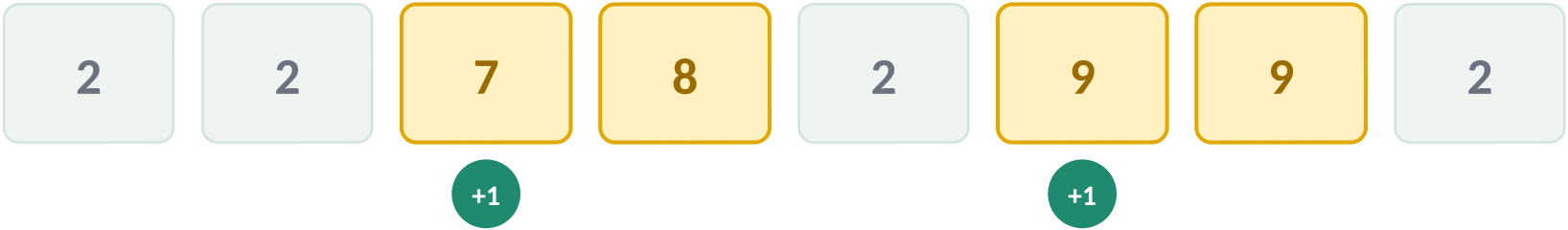
Terminal

```
WRONG: count made every round
  round 1 count is 1
  round 2 count is 1
  round 3 count is 1

RIGHT: count made once
  round 1 count is 1
  round 2 count is 2
  round 3 count is 3
```

No wiring. This one runs even if your circuit is in pieces.

Remembering the LAST one



loud readings: 4

loud moments: 2

The memory, in four lines

```
is_loud = value > LOUD
if is_loud and not was_loud:
    shouts = shouts + 1
was_loud = is_loud
```

`not was_loud` is the whole trick. Count only where quiet turns into loud, not every round it stays loud.

Step 2b: move one wrong line

2b

step2b_find_the_bug.py · lines 14–26

```
14  readings = [1, 2, 8, 9, 1, 7, 1, 1, 6, 6, 2]
15  LOUD = 5
16
17  was_loud = False
18
19  for value in readings:
20      count = 0
21      is_loud = value > LOUD
22      if is_loud and not was_loud:
23          count = count + 1
24      was_loud = is_loud
25
26  print("Loud moments:", count)
```

1. Run it. What does it print?
2. Add a print() inside the loop
3. Which line throws away what it knew?
4. Change ONE line. Run again

It prints 0. The right answer is 3. One line is in the wrong place.

STUCK? Do not rewrite the file. Find the line.

Five minutes, in pairs.

Self-check: state

- [ ] I can say what a program forgets each time round a loop — *slide: a loop forgets*
- [ ] I can say why count = 0 must sit outside the loop — *Step 2, the two Shells*
- [ ] I can say why was_loud is needed to count moments — *slide: remembering the last one*
- [ ] I can find a bug like that in someone else's code — *Step 2b*

Step 3 is unwriteable without these four. Say now if one is missing.

When it does not work

1 Does it run at all?	Red text names a line number. Open that line first.
2 Go back to the last version that worked	Step 2 ran, Step 3 does not? The bug is in what you changed.
3 Change one thing. Then run	Two changes at once and the result teaches you nothing.
4 Cut the program in half	Input, decision, or output? Test each one alone.
5 Make the invisible visible	print() the value you are guessing about.

Starting over is not number 6. It throws away the working half as well.

Where exactly is it wrong?

What you see	The usual cause	Why
The count races up while somebody talks	`not was_loud is missing`	you are counting readings, not moments
The count never moves at all	`was_loud = is_loud is missing`	the memory never updates
The count is stuck at 1	`loud_count = 0 is inside the loop`	it is made again every round
The count climbs in a silent room	`QUIET is too low for this room`	not a bug. Measure your room

Symptoms, not solutions. Match what you see, then go and look at that line yourself.

Step 3: write the counter

3

step3_noise_counter.py · lines 84–97

```
84  is_loud = level >= TOO_LOUD_LEVEL
85  now = time.time()
86
87  # >>>>> YOUR CODE STARTS HERE >>>>>
88  # Line 1: count a new loud moment. Only if the room
89  # was quiet before, and only if COOLDOWN seconds
90  # passed since the last one. Add 1 to loud_count,
91  # then set last_count_time = now.
92  if False:
93      pass
94
95  # Line 2: remember is_loud for next time round.
96
97  # <<<<< YOUR CODE ENDS HERE <<<<<
```

Two lines. The hard part is already decided: count a moment only where quiet turns into loud.

Shout for five seconds. Still 1? Correct.

now time.time() gives seconds

COOLDOWN 2 seconds before a new moment can count

then remember today's answer for next round

CHECKPOINT One clap adds exactly 1.

STUCK? Count racing? was_loud is missing.

What the meter remembers

3

step3_noise_counter.py · lines 70–75

```
70  # ===== 4. THE MEMORY: made once, before the loop =====
71  # The program's memory. Made ONCE, before the loop.
72  loud_count = 0
73  was_loud = False
74  last_count_time = 0
75
```

`loud_count` — how many loud moments so far

`was_loud` — was it loud LAST round?

`last_count_time` — when the last moment was counted

Three boxes, made once

Everything else in the program is worked out fresh each round. These three are the only things that survive from one round to the next.

Predict first: if you delete the COOLDOWN check, what does one clap do?

A clap is not one smooth sound. The level dips in the middle of it, so the room goes loud, quiet, loud, and the count goes up twice. COOLDOWN is what stops that.

Step 4: run it with the LCD

4

step4_lcd_report.py · lines 62–70

```
62  def show(level):
63      for i in range(5):
64          leds[i].value(1 if i < level else 0)
65
66
67  def draw(level, count):
68      """Redraw the LCD. 4 rows, 18 characters each."""
69      lcd.Clear()
70      lcd.Cursor(0, 0)
```

1. Upload **lcd128_32.py** and the fonts file too
2. LCD: SDA to GPIO21, SCL to GPIO22
3. Run it. The screen shows level and count

CHECKPOINT The bar on screen matches the LEDs.

```
Noise level: 3
[###..]
Too loud: 7
```

The LCD from Lesson 1, doing a real job this time.

Only redraw when it changed.
Clearing the screen ten times a second makes it flicker and slows the whole loop.

Say level, not decibels

Honest

"The room reached level 4 eleven times in this lesson."

Not honest

"The room was 85 decibels."

A phone app will disagree with you, and the phone will be right.

Nobody calibrated this sensor against a known sound. It measures a gap that follows loudness. That is enough to compare one lesson with another, and not enough to put a unit on.

Measuring something is easy. Saying exactly what you measured is the hard part, and it is the part people believe.

Ten real minutes

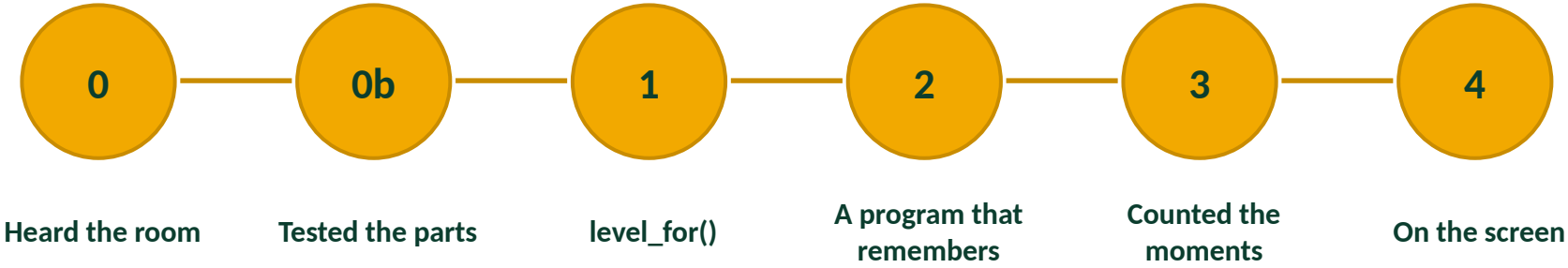
1. Run the meter for the last ten minutes of this lesson
2. Do not tell the class it is running
3. Write the count on the board at the end
4. Write it down. You will need it on 4 November

The number on the board is evidence. On 4 November you pick the problem you will build for, and evidence beats opinions.

Talk to the person next to you

- Who decides what too loud means? You set TOO_LOUD_LEVEL yourself.
- A meter that beeps when the room is loud: quieter room, or louder?
- Who in this school would actually use this? Name a person or a room.
- What would make somebody trust your number?

What did you reach today?



- One reading lies. Fifty readings, loudest minus quietest, tell the truth.
- State is what the program knows because of what happened before. Made once, outside the loop.
- A broken whole is three parts. Test each one alone and the bug has nowhere to hide.

Next, 4 November: pick the problem, and bring the number you measured today.